

vsomeip DTLS Summary

What exists, how the software is structured, how it is verified, and how to review and test it. For reviewers, testers and integrators evaluating the `test/dtls-udp` branch.

DOCUMENT ID	VSOMEIP-DTLS-SUM	VERSION	1.0 · Draft for community review
AUTHOR	duonghvu	DATE	17 Sep 2026
BRANCH	duonghvu/vsomeip · test/dtls-udp	BASE	COVESA vsomeip 3.7.5 (7875a1cb) + merge of feature/dtls-udp (3.7.2-based, 39066aa6)
COMPANION	vsomeip_DTLS_Requirements_Specification.pdf: requirements, implementation steps, open issues		

1 At a glance

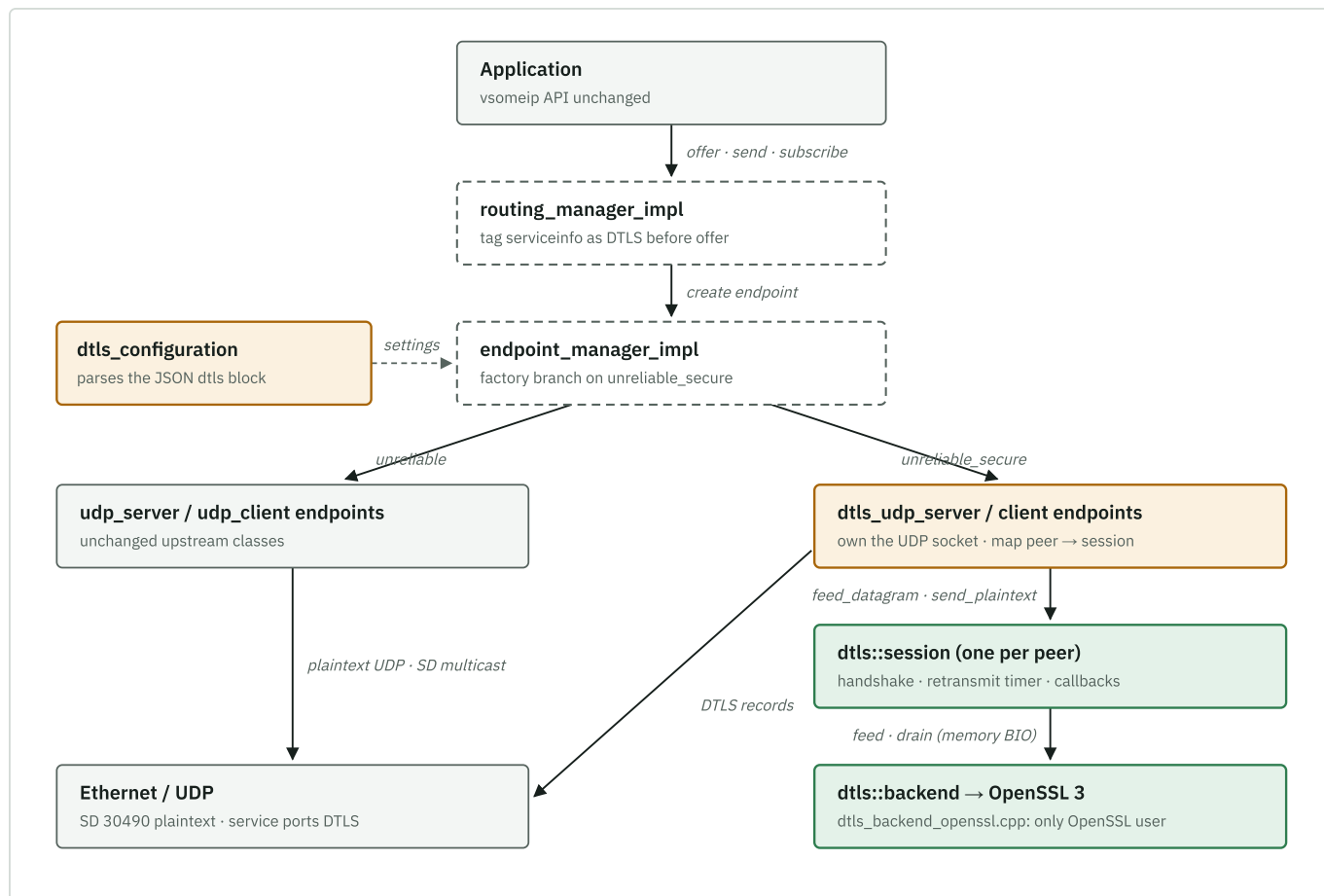
SOME/IP services can be protected with mutually authenticated DTLS 1.2 by configuration alone: a service sets `"unreliable_secure": <port>` in the JSON config and application code doesn't change. The crypto lives in the library, behind a backend interface (OpenSSL 3 first), in new classes next to the existing UDP endpoints. Service discovery multicast stays plaintext.

● Crypto backend (OpenSSL 3) Built; handshake, mutual auth, data and close_notify pass in the standalone simulation	● Per-peer DTLS session Built; retransmit timer and callbacks exercised by the simulation	● DTLS server / client endpoints Written, never compiled; must be adapted to the 3.7.5 endpoint API
● JSON dtls parser Written; not yet called from configuration_impl	○ Factory, routing and config wiring Not started: DTLS endpoints are never created at runtime yet	○ Unit, integration and network tests Not started; only the standalone simulation exists
● Cookie exchange Implemented incorrectly: cookie isn't tied to the client address (see §7)	● Build switch <code>ENABLE_DTLS</code>, default OFF; all DTLS code behind <code>VSOMEIP_HAS_DTLS</code>	

Maturity: experimental. The branch shows the design and the verified crypto core, and asks for early feedback. It is not ready for production or for an upstream pull request.

2 Architecture

Five architecture decisions shape the code. DTLS lives in **new classes alongside** the UDP endpoints, which stay unmodified. **OpenSSL is reached only through `dtls::backend`**, so Mbed TLS or an HSM-backed library can replace it. There is **one session per remote IP and port**, DTLS is **opt-in per service**, and **SD multicast stays plaintext**.



 Upstream, unchanged
 New, verified in simulation
 New, not yet compiled or wired
 Planned change

The endpoint layer is the only place sockets and crypto meet. The session never touches the socket: the endpoint passes datagrams in, and the session hands ciphertext and plaintext back through callbacks. That is also what lets the crypto core be tested without a network.

3 Software file structure

Every DTLS file on `test/dtls-udp`, plus the files the remaining work will add or change. Line counts are from the current branch.

PATH	LINES	STATE	PURPOSE
vsomeip/			
─ CMakeLists.txt	+20	modified	ENABLE_DTLS option, OpenSSL ≥ 3.0 lookup and link
─ README.md		modified	DTLS guidelines and community review section
─ DTLS_TESTING.md	263	added	Operator test guide (Ubuntu), production gate
─ implementation/endpoints/include/			
─ dtls_backend.hpp	96	verified	backend, context, session_state interfaces; result_e
─ dtls_session.hpp	101	verified	Per-peer session: handshake driver, timer, callbacks
─ dtls_udp_server_endpoint_impl.hpp	97	not compiled	Server endpoint: socket, peer → session map
─ dtls_udp_client_endpoint_impl.hpp	75	not compiled	Client endpoint: one session, queue until handshake
─ implementation/endpoints/src/			
─ dtls_backend_openssl.cpp	419	verified	OpenSSL 3 context, verify, cipher pin, cookies, memory BIO
─ dtls_session.cpp	192	verified	Handshake pump, retransmit timer, shutdown
─ dtls_udp_server_endpoint_impl.cpp	285	not compiled	Receive, route to session, decrypt, deliver upward
─ dtls_udp_client_endpoint_impl.cpp	203	not compiled	Connect, handshake, flush queue
─ endpoint_manager_impl.cpp		planned	Create DTLS endpoints for unreliable_secure
─ implementation/configuration/			
─ include/dtls_configuration.hpp	41	not wired	Parsed dtls block, secure port map
─ src/dtls_configuration.cpp	65	not wired	JSON parser with auto-disable rules
─ include/configuration_impl.hpp		planned	DTLS getters
─ src/configuration_impl.cpp		planned	load_dtls, per-service unreliable_secure

PATH	LINES	STATE	PURPOSE
└ implementation/routing/src/routing_manager_impl.cpp		planned	Tag serviceinfo during preparation stage
└ examples/dtls_simulation/		verified	Standalone round trip, no libvsomeip3
└ CMakeLists.txt	60		Builds only backend + session into two nodes
└ dtls_server_node.cpp	188		Echo server
└ dtls_client_node.cpp	242		Ping client, PASS/FAIL exit code
└ gen_test_certs.sh	43		Test CA + server/client ECDSA P-256 certs
└ run_simulation.sh	251		Build, run, render result.txt / result.svg
└ readme.md	60		How to run
└ examples/hello_world_dtls/		planned	hello_world over DTLS with config only
└ test/unit_tests/dtls_tests/		planned	unit_dtls_* : backend, session, config
└ test/network_tests/dtls_tests/		planned	Two-host, interop, stress, failure cases
└ documentation/dtls/		added	This summary and the requirements specification (HTML + PDF)

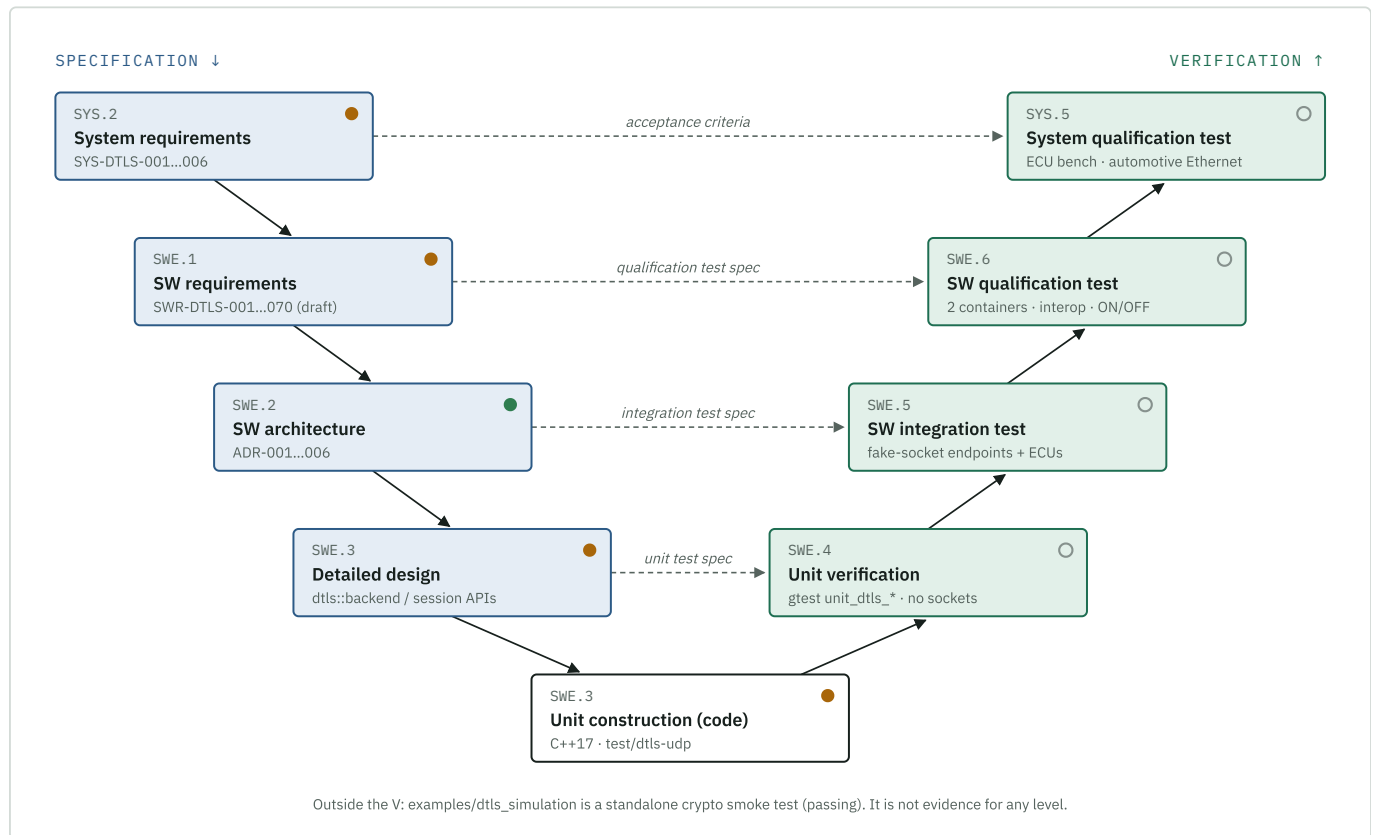
4 Configuration

```
{
  "dtls": {
    "enable": "true",
    "version": "1.2",
    "cipher_list": "ECDHE-ECDSA-AES128-GCM-SHA256",
    "certificate": "/etc/vsomeip/ecu.pem",
    "private_key": "/etc/vsomeip/ecu.key",
    "ca_bundle": "/etc/vsomeip/ca.pem",
    "verify_peer": "true",
    "handshake_timeout_ms": 5000,
    "cookie_exchange": "true"
  },
  "services": [
    { "service": "0x2444", "instance": "0x0001",
      "unreliable_secure": "32444" }
  ]
}
```

- **Additive only.** Existing keys keep their meaning; `unreliable` still means plaintext UDP.
- **Mutual auth by default.** With `verify_peer` on, a missing `ca_bundle` disables DTLS instead of silently accepting any peer.
- **Missing certificate or key** also disables DTLS.
- **Transition:** `unreliable` and `unreliable_secure` together are allowed only while peers migrate, and should log a warning.
- **Not yet active:** the parser exists but `configuration_impl` doesn't call it, and `handshake_timeout_ms` is parsed but not enforced.

5 Verification approach

Each left-side level of the V-model has a matching test level. Test specs are written together with the level they verify. Process IDs follow Automotive SPICE 3.1.



● Done ● Partial ○ Not started

LEVEL	WHAT IS EXERCISED	WHERE	RUNS ON
Unit · SWE.4	Backend and session through in-memory byte passing: handshake, wrong CA, expired cert, loss and reordering with <code>handle_timeout()</code> , config parser	<code>test/unit_tests/dtls_tests/</code>	macOS, Linux, CI (<code>^unit_</code>)
Component · SWE.5	One DTLS endpoint with injected fake sockets: session lifecycle, queue flush, <code>close_notify</code> on stop	<code>test/unit_tests/endpoint_tests/</code>	Linux, CI
Multi-ECU · SWE.5	Config → routing → SD → endpoints → crypto for two ECUs in one process; drop, delay, disconnect; hybrid mode	<code>test/network_tests/fake_socket_tests/</code>	Linux, CI
Qualification · SWE.6	Real processes and sockets on two containers; interop with <code>openssl s_client/s_server</code> ; <code>tcpdump</code> ; 1000-handshake stress; <code>ENABLE_DTLS</code> ON and OFF	<code>test/network_tests/dtls_tests/</code> , <code>zuul/network-tests</code>	Linux containers, nightly
System · SYS.5	Target ECUs: Ethernet MTU, OEM PKI, revocation, peer reboot, CPU and latency budget	Bench	Integration bench

What the simulation proves

- OpenSSL 3 context, ECDSA certificates, CA chain check

What it does not prove

- That the vsomeip endpoints compile or work

- Full DTLS 1.2 handshake with HelloVerifyRequest
- Mutual authentication with the pinned cipher
- Encrypted data both ways; graceful close_notify
- Config → factory → endpoint creation; SD → connect → handshake
- SOME/IP messages and TP over DTLS; loss, many peers, MTU
- Failure cases, hybrid mode, OFF build identical to upstream

6 Try it

Standalone simulation (macOS or Linux)

```
git clone -b test/dtls-udp https://github.com/duong
cd vsomeip/examples/dtls_simulation
./run_simulation.sh --count 5
# expect: DTLS simulation: PASS
# outputs: build/result.txt, build/result.svg
```

Full library (Linux, or the repo Dockerfile)

```
sudo apt-get install -y libssl-dev libboost-all-dev
cmake -Bbuild -DENABLE_DTLS=OFF -DGTEST_ROOT=/usr/s
cmake --build build --target build_tests -j
ctest --test-dir build -R '^unit_' --output-on-fail
# ENABLE_DTLS=ON fails to compile today (see §7)
```

7 Known issues from code review

Found by reading the code on 16–17 Sep 2026. None has been fixed yet. The first two block any meaningful community test of the full library.

SEVERITY	ISSUE	LOCATION	CONSEQUENCE
High	The cookie is a copy of one process-wide secret, not an HMAC of the client address (the code comment says it is). If <code>RAND_bytes</code> fails, the secret falls back to predictable bytes.	<code>dtls_backend_openssl.cpp:33–67</code>	Anyone who gets one cookie can use it from any spoofed address, so the cookie exchange gives no DoS or amplification protection.
High	DTLS endpoints don't compile: the server calls <code>asio_socket_factory::create_udp_socket(io_, _error)</code> as a static function, and both endpoints override the pre-3.7.5 <code>tp_segmentation_enabled(service_t, instance_t, method_t)</code> .	<code>dtls_udp_server_endpoint_impl.cpp:36;</code> <code>*_endpoint_impl.hpp</code>	<code>ENABLE_DTLS=ON</code> fails to build the library.
Medium	Any new source address creates a session; there is no limit and no idle eviction.	<code>dtls_udp_server_endpoint_impl.cpp</code>	Memory exhaustion from spoofed traffic.
Medium	Session callbacks capture <code>shared_from_this()</code> , creating endpoint → session → callback → endpoint cycles.	both endpoint .cpp files	Endpoints and sessions are never freed until stop or restart.
Medium	<code>handshake_timeout_ms</code> is parsed but never enforced; the link MTU is hard-coded to 1500.	<code>dtls_backend_openssl.cpp:200</code>	Stalled handshakes hold state; smaller-MTU networks may fragment or fail.
Medium	The server can send only to peers that already completed a handshake.	<code>dtls_udp_server_endpoint_impl.cpp</code>	Events can't reach subscribers without an established session.
Low	<code>cipher_list</code> uses OpenSSL syntax, and the server session factory isn't given the peer address.	<code>dtls_backend.hpp</code>	Blocks a second backend (Mbed TLS) and a correct cookie design.
Low	The minimum is OpenSSL 3.0, which reached end of life in Sep 2026.	<code>CMakeLists.txt</code>	Recommend 3.5 LTS as the documented baseline.

8 Asking the vsomeip community to review and test

COVESA hosts vsomeip. Its GitHub repository has Discussions and Issues enabled, and the project holds a monthly community call (third Monday of the month, 16:00 CET). COVESA mailing lists and Slack access are requested through the COVESA wiki.

1. **Fix the two High issues first.** Reviewers lose interest quickly in code that doesn't build, and a broken cookie design will be the first security comment.
2. **Open a GitHub Discussion, not a pull request.** Post an RFC in COVESA/vsomeip Discussions, link `test/dtls-udp` and both PDFs, and label it experimental.
3. **Ask concrete questions.** How clients learn a remote service needs DTLS, fail-closed vs fallback, server-initiated events. Maintainers answer specific design questions faster than "please review".
4. **Give testers a recipe and a report format.** The simulation command, the build ON/OFF commands, and: OS, compiler, Boost, OpenSSL version, command, result, logs.
5. **Ask for a slot at the monthly community call** to present the design (10 minutes, using this summary).
6. **Split the upstream contribution** once feedback is in and tests are green: (1) backend + session + unit tests, (2) endpoints, (3) config and factory wiring, (4) example and docs. Each as a PR against `master`, following CONTRIBUTING.md (clang-format pre-commit hook).

DRAFT DISCUSSION POST

RFC: opt-in DTLS 1.2 over UDP for vsomeip (experimental branch, feedback wanted)

We are prototyping native DTLS 1.2 for unicast SOME/IP over UDP, enabled per service by config (`"unreliable_secure": <port>`) with no application changes. SD multicast stays plaintext. DTLS lives in new endpoint classes next to the UDP endpoints, with crypto behind a backend interface (OpenSSL 3 first). It is compiled only with `-DENABLE_DTLS=ON` .

Branch: `duonghvu/vsomeip@test/dtls-udp` (vsomeip 3.7.5 base). Summary and requirements: `documentation/dtls/` . Status: crypto core verified in a standalone simulation; library wiring and tests not done.

Questions: (1) Would maintainers accept this direction upstream, and in what PR split? (2) Should the client learn that a remote instance needs DTLS from local config only, or is an SD option acceptable? (3) Fail closed when DTLS is misconfigured, or fall back? (4) Preferred way for the server to reach subscribers before they complete a handshake?

Testers: please run `examples/dtls_simulation/run_simulation.sh` and report OS, compiler, Boost and OpenSSL versions with the result.

Author: duonghvu · Drafted with Claude Code assistance · Sources: vsomeip_DTLS_UDP_3_7_2_Plan.md rev 0.2, code on test/dtls-udp, COVESA vsomeip 3.7.5 · Licence of this document follows the repository (MPL-2.0).